



Passo 1 de 5: Organização do Repositório

Tudo dentro de `www/` + um workflow

Toda a aplicação web fica em `www/`. O GitHub Pages publica essa pasta como PWA, e o Capacitor (acionado pelo workflow) empacota a mesma pasta como APK. Sem duplicar código.

Estrutura do repositório:

```
seu-repo/
├── .github/
│   └── workflows/
│       └── build.yml (workflow)
└── www/ <-- tudo do app aqui
    ├── index.html
    ├── style.css
    ├── app.js
    ├── manifest.json (PWA)
    ├── sw.js (Service Worker)
    └── icons/
        ├── icon-192.png
        └── icon-512.png
```

Passo 2 de 5: CRUD de Alunos

Cadastro com Nome, E-mail e Idade

O formulário salva no `LocalStorage`. O mesmo código roda no navegador (PWA) e dentro do WebView do APK.

⚠️ Atenção ao copiar: use sempre o botão **Copiar** de cada bloco. Não copie manualmente do texto na tela — o `app.js` usa aspas e chaves que se quebram facilmente ao copiar à mão.

Visualização Prévia

• LocalStorage em tempo real



AlunosApp
Gestão Escolar

Nome Completo

E-mail

Idade

Salvar Aluno

ID	NOME	IDADE	AÇÕES
----	------	-------	-------

Nenhum aluno cadastrado.

index.html

style.css

app.js

 Copiar index.html

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0, maximum-scale=1.0, user-scalable=no">
  <title>AlunosApp</title>
  <link rel="stylesheet" href="style.css">
  <link rel="manifest" href="manifest.json">
  <meta name="theme-color" content="#0284c7">
  <link rel="icon" type="image/png" sizes="192x192" href="icons/icon-192.png">
</head>
<body>
  <div class="container">
    <header class="topo">
      <div class="icone">🎓</div>
      <div>
        <h1>AlunosApp</h1>
        <p>Gestão Escolar</p>
      </div>
    </header>
```

Passo 3 de 5: Instalável no navegador **O que é PWA**

Com **manifest.json** + **sw.js**, o Chrome (Android ou desktop) oferece a opção "*Instalar aplicativo*" ou "*Adicionar à tela inicial*". O app abre em tela cheia como um nativo. O mesmo código servirá para o APK.

 www/manifest.json Copiar manifest.json

```
{
  "name": "AlunosApp - Gestão Escolar",
  "short_name": "AlunosApp",
  "description": "App de cadastro de alunos com LocalStorage",
  "start_url": "./index.html",
  "scope": "./",
  "display": "standalone",
  "orientation": "portrait",
  "background_color": "#0f172a",
  "theme_color": "#0284c7",
  "icons": [
    {
      "src": "icons/icon-192.png",
      "sizes": "192x192",
      "type": "image/png",
      "purpose": "any"
    },
    {
      "src": "icons/icon-512.png",
      "sizes": "512x512",
```

⚙️ www/sw.js (Service Worker)

 Copiar sw.js

```
const CACHE = 'alunosapp-v2';
const ASSETS = [
  './',
  './index.html',
  './style.css',
  './app.js',
  './manifest.json',
  './icons/icon-192.png',
  './icons/icon-512.png'
];

self.addEventListener('install', e => {
  e.waitUntil(caches.open(CACHE).then(c => c.addAll(ASSETS)));
  self.skipWaiting();
});

self.addEventListener('activate', e => {
  e.waitUntil(
    caches.keys().then(keys =>
      Promise.all(keys.filter(k => k !== CACHE).map(k => caches.delete(k)))
    )
  );
});
```

⚠️ A cada nova versão do site, troque `alunosapp-v2` por `alunosapp-v3`, `-v4`, etc. Isso força o service worker a descartar o cache antigo e baixar os arquivos novos. **Sem isso, o celular continua mostrando a versão antiga.**

Os arquivos `icons/icon-192.png` e `icons/icon-512.png` precisam ser **PNG reais** (upload, não texto). Gere em favicon.io ou realfavicongenerator.net. Sem eles, o aviso de instalação PWA não aparece.

Passo 4 de 5: Gera APK e publica PWA

⚙️ .github/workflows/build.yml

Este arquivo faz duas coisas: **(1)** publica `www/` no GitHub Pages a cada push em `main`; **(2)** gera o APK com Capacitor e anexa numa **Release** quando você cria uma tag `v*`. O APK fica disponível por HTTPS no github.com, sem bloqueio do Chrome.

 Copiar build.yml

```
name: Deploy PWA & Build APK

on:
  push:
    branches: [main]
    tags: ['v*']
  workflow_dispatch:

permissions:
  contents: write
  pages: write
  id-token: write
```

```
jobs:
  # ===== PWA (GitHub Pages) =====
  deploy-pwa:
    runs-on: ubuntu-latest
    environment:
      name: github-pages
      url: ${ steps.deployment.outputs.page_url }
    steps:
```

Como o workflow roda:

- **Push em main** → job **deploy-pwa** publica o site no Pages.
- **Push em main** → job **build-apk** também roda, gerando o APK nos *Artifacts*.
- **Tag v1.0.0** → o APK também é anexado à *Release*, gerando link HTTPS público.

🔥 **Erro mais comum:** **Error: Get Pages site failed** . Isso significa que o GitHub Pages ainda não foi ativado no repositório. Vá em *Settings* → *Pages* → *Source: GitHub Actions* e rode o workflow de novo.

Passo 5 de 5: 100% pelo navegador

👤 Passo a passo

- 1 **Crie um repositório novo** no GitHub, por exemplo **alunosapp** , marcado como **Public**.
- 2 **Crie os arquivos de texto** clicando em "Add file → Create new file". No campo do nome, digite o caminho completo. O GitHub cria as pastas automaticamente ao digitar `/` :

Ordem sugerida de criação:
 1. `www/index.html`
 2. `www/style.css`
 3. `www/app.js`
 4. `www/manifest.json`
 5. `www/sw.js`
 6. `.github/workflows/build.yml`
- 3 **Suba os ícones PNG** com "Add file → Upload files". Envie `icon-192.png` e `icon-512.png` para dentro de `www/icons/` . Precisam ser arquivos PNG reais, não texto.
- 4 **Ative o GitHub Pages:** *Settings* → *Pages* → *Source: GitHub Actions*. Não escolha "Deploy from a branch".
- 5 **Aguarde o build** na aba *Actions*. Após ~1 minuto, o site estará em `https://SEU-USUARIO.github.io/alunosapp/` . Abra no Chrome do celular → aparecerá "Adicionar à tela inicial".
- 6 **Para gerar o APK:** vá em *Releases* → *Create a new release*, crie uma tag `v1.0.0` , publique. O workflow rodará automaticamente e anexará o APK na Release.
- 7 **Baixe o APK** pelo link `https://github.com/SEU-USUARIO/alunosapp/releases/download/v1.0.0/app-debug.apk` . É HTTPS do github.com, então o Chrome não bloqueia.

✅ Nada foi instalado no seu computador. Todo o build e a publicação rodam nos servidores do GitHub.

⚠️ O APK gerado é de **debug** — instala direto no celular, perfeito para testes. Para publicar na Play Store, seria preciso assinar um APK de *release* com keystore própria (fora do escopo desta aula).

🔔 **Celular mostrando versão antiga do site?** O *Service Worker* guarda os arquivos em cache. Para forçar a atualização:

1. **Mude a versão no `sw.js`** : troque `alunosapp-v2` por `alunosapp-v3` .
2. Faça commit e rode o workflow.
3. No celular, no Chrome: toque no cadeado 🔒 → *Cookies e dados do site* → *Excluir dados*.
4. Recarregue o site.

Resumo Geral

TECNOLOGIA	LINGUAGEM	COMO GERA O APP?	QUANDO USAR?
Capacitor	HTML / CSS / JS	Embrulha <code>www/</code> em WebView nativa e gera APK	Aproveitar um sistema web existente e transformá-lo em app.
PWA	HTML / CSS / JS	Instalação direta pelo navegador, sem loja	Sistemas rápidos que não precisam estar na Play Store.
React Native	JavaScript / React	Compila componentes nativos do Android e iOS	Aplicações comerciais de grande porte com equipe JS.
Flutter	Dart	Renderiza com motor próprio (Skia/Impeller)	Apps com animações complexas e design idêntico em tudo.
Kotlin	Kotlin (JVM)	Desenvolvimento Android nativo oficial	Apps com alto processamento gráfico, sensores e Bluetooth.